

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«КУРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Факультет физики, математики, информатики

Кафедра программного обеспечения и администрирования информационных систем

Отчет
по лабораторной работе №1
«Объекты и классы»
по дисциплине
«Технологии программирования»

Обучающегося 3 курса очной формы обучения

Направления подготовки

09.03.01 Информатика и вычислительная техника

Направленность (профиль)

Прикладной искусственный интеллект

Жаанбаев Б. К.

Руководитель:

ассистент кафедры ПОАИС

Сазонова К.А.

Курск, 2025

Содержание

Введение	3
Задание в рамках курсового проектирования	4
Требования к заданию в рамках курсового проектирования	4
Выполнение задания в рамках курсового проектирования.....	4
Тестирование программы	5
Индивидуальное задание.....	6
Требования к индивидуальному заданию	6
Выполнение индивидуального задания.....	7
Заключение	9

Введение

Цель работы: освоить механизмы создания и уничтожения объектов в C++ путём разработки класса с использованием всех типов конструкторов (по умолчанию, с параметрами, копирования), списка инициализаторов, указателя `this` и деструктора, а также продемонстрировать работу с объектами в различных контекстах (статические, динамические, массивы, с явным и неявным вызовом деструктора).

Задание в рамках курсового проектирования

Требования к заданию в рамках курсового проектирования

Разработать класс и программу, в которой создаются и разрушаются объекты класса (при использовании всех типов конструкторов) для 5 варианта – математический анализ

Требования к классу:

- наличие в классе всех необходимых полей для хранения данных;
- наличие закрытой (private) и общедоступной (public) частей;
- класс должен обеспечивать все необходимые способы создания объектов (при этом использовать все типы конструкторов);
- хотя бы в одном из конструкторов необходимо использовать список инициализаторов;
- класс должен иметь деструктор.

Выполнение задания в рамках курсового проектирования

Класс Piece (шахматная фигура)

Класс представляет шахматную фигуру на доске. Он содержит пять закрытых полей:

- type — символ (char) — тип фигуры ('P', 'R', 'N', 'B', 'Q', 'K').
- row, col — целые числа (int) — координаты на доске (0–7).
- color — перечисляемый тип (enum class) — цвет фигуры (White / Black).
- name — строка (std::string) — полное название (например, "White Queen").

Публичная часть включает:

- Три конструктора:

1. По умолчанию — создаёт белую пешку на позиции (0,0). Использует явное this при инициализации.
 2. С параметрами — принимает тип, позицию и цвет. Все поля инициализируются через список инициализаторов. Название формируется автоматически.
 3. Копирования — копирует все данные, добавляя к названию пометку "(копия)".
- Деструктор — выводит сообщение об удалении фигуры с её полным именем.
 - Метод show() — отображает тип, позицию, цвет и название фигуры.

Текст программы приведен в приложении А.

Тестирование программы

Тестирование программы представлено на рисунке 1.

```
Конструктор копирования: Black Queen (копия)
Фигура: Black Queen (копия), Позиция: (3,4), Цвет: Чёрный

--- Массив из 2 фигур ---
Конструктор по умолчанию: White Pawn
Конструктор по умолчанию: White Pawn
Фигура: White Pawn, Позиция: (0,0), Цвет: Белый
Фигура: White Pawn, Позиция: (0,0), Цвет: Белый

--- Динамическая фигура ---
Конструктор с параметрами: White King at (7,4)
Фигура: White King, Позиция: (7,4), Цвет: Белый
Деструктор: удалена фигура 'White King'

--- Временная фигура в блоке ---
Конструктор с параметрами: White Rook at (0,0)
Фигура: White Rook, Позиция: (0,0), Цвет: Белый
Деструктор: удалена фигура 'White Rook'

=== Конец программы ===
Деструктор: удалена фигура 'White Pawn'
Деструктор: удалена фигура 'White Pawn'
Деструктор: удалена фигура 'Black Queen (копия)'
Деструктор: удалена фигура 'Black Queen'
Деструктор: удалена фигура 'White Pawn'
```

Рисунок 1.

Индивидуальное задание

Требования к индивидуальному заданию

В соответствии с вариантом задания самостоятельно разработать класс и программу, в которой создаются и разрушаются объекты класса (при использовании всех типов конструкторов). В программе предусмотреть использование массива объектов класса. Продемонстрировать в программе примеры явного и неявного вызова деструкторов: для статических и динамических объектов класса.

Требования к классу:

- наличие как минимум трех полей класса разных типов данных, один из которых должен быть перечисляемым;
- наличие закрытой (private) и общедоступной (public) частей;
- класс должен иметь три типа конструкторов, определенных программистом:
- конструктор без параметров, конструктор с параметрами и конструктор копии;
- хотя бы в одном из конструкторов необходимо использовать список
- инициализаторов;
- хотя бы в одном из конструкторов необходимо использовать указатель this в явном виде;
- вызов каждого из конструкторов должен сопровождаться выдачей соответствующего сообщения;
- класс должен иметь деструктор.

Выполнение индивидуального задания

Класс `MathematicalAnalysis` (вариант 5 — Математический анализ)

Класс разработан для моделирования учебного курса по математическому анализу. Он содержит четыре закрытых поля различных типов:

- `name` — строка (`std::string`) — название курса.
- `level` — перечисляемый тип (`enum class`) — уровень сложности (Лёгкий, Средний, Сложный).
- `topics` — целое число (`int`) — количество тем в курсе.
- `credits` — вещественное число (`float`) — количество зачётных единиц.

Публичная часть включает:

- Три конструктора:
 1. По умолчанию — инициализирует объект стандартными значениями (например, "Математический анализ", лёгкий уровень, 0 тем, 3 кредита). Использует явное указание `this` при присваивании полям.
 2. С параметрами — принимает все данные (название, уровень, количество тем, кредиты). Инициализация полей выполняется через список инициализаторов.
 3. Копирования — создаёт копию существующего объекта, добавляя к названию пометку "(копия)".
- Деструктор — выводит сообщение об уничтожении объекта с указанием его названия.
- Метод `show()` — выводит полную информацию о курсе в удобочитаемом виде.

Текст программы приведен в приложении Б.

Тестирование программы

Тестирование программы представлено на рисунке 2.

```
=== Начало программы ===  
Конструктор без параметров  
Название: Math Basics, Уровень: Easy, Тем: 0  
Конструктор с параметрами  
Название: Calculus, Уровень: Medium, Тем: 10  
Конструктор копирования  
Название: Calculus, Уровень: Medium, Тем: 10  
Конструктор без параметров  
Конструктор без параметров  
Название: Math Basics, Уровень: Easy, Тем: 0  
Название: Math Basics, Уровень: Easy, Тем: 0  
Конструктор с параметрами  
Название: Advanced Math, Уровень: Hard, Тем: 15  
Деструктор: Advanced Math  
Конец программы  
Деструктор: Math Basics  
Деструктор: Math Basics  
Деструктор: Calculus  
Деструктор: Calculus  
Деструктор: Math Basics
```

Рисунок 2.

Заключение

Вывод: освоили механизмы создания и уничтожения объектов в C++ путём разработки класса с использованием всех типов конструкторов (по умолчанию, с параметрами, копирования), списка инициализаторов, указателя `this` и деструктора, а также продемонстрировали работу с объектами в различных контекстах (статические, динамические, массивы, с явным и неявным вызовом деструктора).

ПРИЛОЖЕНИЕ А

```
#include <iostream>
#include <string>
#include <io.h>
#include <fcntl.h>

enum class Color { White, Black }; // Перечисляемый тип — один
из трёх разных типов

class Piece
{
private:
    char type;           // 'P', 'R', 'N', 'B', 'Q', 'K'
    int row;             // строка (0-7)
    int col;             // столбец (0-7)
    Color color;         // цвет: White / Black
    std::string name;    // полное имя фигуры

public:
    // 1. Конструктор по умолчанию — используем this явно
    Piece() {
        this->type = 'P';
        this->row = 0;
        this->col = 0;
        this->color = Color::White;
        this->name = "White Pawn";
        std::cout << "Конструктор по умолчанию: " << name <<
std::endl;
    }

    // 2. Конструктор с параметрами — СПИСОК ИНИЦИАЛИЗАТОРОВ
    Piece(char t, int r, int c, Color clr)
```

```

        : type(t), row(r), col(c), color(clr) // <-- список
инициализаторов
    {
        // Формируем имя
        std::string colorStr = (clr == Color::White) ? "White "
: "Black ";
        switch (std::tolower(t)) {
            case 'p': name = colorStr + "Pawn"; break;
            case 'r': name = colorStr + "Rook"; break;
            case 'n': name = colorStr + "Knight"; break;
            case 'b': name = colorStr + "Bishop"; break;
            case 'q': name = colorStr + "Queen"; break;
            case 'k': name = colorStr + "King"; break;
            default: name = colorStr + "Unknown"; break;
        }
        std::cout << "Конструктор с параметрами: " << name << "
at (" << row << ", " << col << ")" << std::endl;
    }

// 3. Конструктор копирования
Piece(const Piece& other)
{
    type = other.type;
    row = other.row;
    col = other.col;
    color = other.color;
    name = other.name + " (копия)";
    std::cout << "Конструктор копирования: " << name <<
std::endl;
}

// Деструктор
~Piece() {
    std::cout << "Деструктор: удалена фигура '" << name <<
"' " << std::endl;
}

```

```

    }

    // Метод для вывода (демонстрация)
    void show() const
    {
        std::cout << "Фигура: " << name
            << ", Позиция: (" << row << ", " << col << ")"
            << ", Цвет: " << (color == Color::White ? "Белый" :
"Чёрный")
            << std::endl;
    }
};

int main()
{
    // Настройка консоли для русского текста
    setlocale(LC_ALL, "RUS");

    std::wcout << L"=== Лабораторная работа: класс Piece
===\n\n";

    // 1. Объект по умолчанию
    Piece p1;
    p1.show();

    // 2. Объект с параметрами
    Piece p2('Q', 3, 4, Color::Black);
    p2.show();

    // 3. Копирование
    Piece p3 = p2;
    p3.show();

    // 4. Массив объектов

```

```

std::wcout << L"\n--- Массив из 2 фигур ---\n";
Piece board[2];
for (int i = 0; i < 2; ++i)
{
    board[i].show();
}

// 5. Динамический объект
std::wcout << L"\n--- Динамическая фигура ---\n";
Piece* pDynamic = new Piece('K', 7, 4, Color::White);
pDynamic->show();
delete pDynamic; // Явный вызов деструктора

// 6. Неявный деструктор в блоке
{
    std::wcout << L"\n--- Временная фигура в блоке ---\n";
    Piece temp('R', 0, 0, Color::White);
    temp.show();
} // ← деструктор автоматически

std::wcout << L"\n=== Конец программы ===\n";
return 0;
}

```

ПРИЛОЖЕНИЕ Б

```
#include <string>
#include <iostream>
#include <io.h>
#include <fcntl.h>

enum Level { Easy, Medium, Hard }; // Перечисление для уровня

class MathAnalysis
{
private:
    std::string name; // Название
    Level level;      // Уровень
    int topics;       // Количество тем

public:
    // Конструктор без параметров
    MathAnalysis()
    {
        this->name = "Math Basics";
        this->level = Level::Easy;;
        this->topics = 0;

        std::cout << "Конструктор без параметров" << std::endl;
    }

    // Конструктор с параметрами - с использованием списка
    инициализации
    MathAnalysis(std::string n, Level l, int t) : name(n),
    level(l), topics(t)
    {
        std::cout << "Конструктор с параметрами" << std::endl;
    }
}
```

```

// Конструктор копирования
MathAnalysis(const MathAnalysis& other)
{
    name = other.name;
    level = other.level;
    topics = other.topics;
    std::cout << "Конструктор копирования" << std::endl;
}

// Деструктор
~MathAnalysis()
{
    std::cout << "Деструктор: " << name << std::endl;
}

// Показать информацию
void show()
{
    std::cout << "Название: " << name << ", Уровень: ";
    if (level == Easy) std::cout << "Easy";
    else if (level == Medium) std::cout << "Medium";
    else std::cout << "Hard";
    std::cout << ", Тем: " << topics << std::endl;
}
};

int main(){
    setlocale(LC_ALL, "RUS");
    std::cout << "=== Начало программы ===" << std::endl;

    // Создаём объект по умолчанию
    MathAnalysis obj1;

```

```
obj1.show();

// Создаём объект с параметрами
MathAnalysis obj2("Calculus", Medium, 10);
obj2.show();

// Копируем объект
MathAnalysis obj3 = obj2;
obj3.show();

// Массив объектов
MathAnalysis arr[2]; // Два объекта по умолчанию
for (int i = 0; i < 2; i++)
{
    arr[i].show();
}

// Динамический объект
MathAnalysis* obj4 = new MathAnalysis("Advanced Math", Hard,
15);
obj4->show();
delete obj4; // Удаляем динамический объект
std::cout << "Конец программы" << std::endl;
return 0;
}
```